

A User Interface for Sense-making of the Reasoning Process while Interacting with Robots

Chao Wang

chao.wang@honda-ri.de

Honda Research Institute Europe
Offenbach am Main, Hessen, Germany

Pengcheng An

School of Design, Southern University of Science and
Technology
Shenzhen, China

Joerg Deigmoeller

Honda Research Institute Europe
Offenbach am Main, Hessen, Germany

Julian Eggert

Honda Research Institute Europe
Offenbach am Main, Hessen, Germany

ABSTRACT

This paper describes an interface that enables experts to communicate with a virtual robot in a simulated environment via natural language, and to visualize the robot's knowledge representation for them for inspection and correction. The interface visually links the robot's internal reasoning processes and knowledge with the simulated instances in the form of a 3D isometric visualization as well as the robot's first-person view. After 3 weeks of using the system by the roboticists in their daily development, some feedback was collected that provided insights for designing such systems in the future.

CCS CONCEPTS

• Human-centered computing → Visualization toolkits.

KEYWORDS

graph representation, human-robot interaction, explainable AI, data visualization

ACM Reference Format:

Chao Wang, Joerg Deigmoeller, Pengcheng An, and Julian Eggert. 2023. A User Interface for Sense-making of the Reasoning Process while Interacting with Robots. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3544549.3585886>

1 INTRODUCTION

In the recent years, robots acting in simulated home environments by combining commonsense knowledge and observations, draws more and more attention in the robotics research community [5, 14]. Nevertheless, most of the works usually miss an understanding of the situation and contextual setting, because it requires a well defined representation of an agent's knowledge. One promising form of such a representation are knowledge graphs, which allow and

incremental growing of the knowledge during operation by accessing external knowledge sources or through interaction with the user. Another important advantage of graph representations is their transparency. As graph representations can be human and machine readable at the same time, they provide a meaningful and insightful explanation of how systems reason [18]. Therefore, the visualization of graphs have been investigated extensively for helping researchers and developers to make sense of such data format [11, 21, 24]. The challenge is still to communicate efficiently and avoid huge amount of information the user has to parse. This requires a good understanding of the current context to focus on most relevant processing steps and data only. There is also a huge potential to use similar visualization methods for helping developers to improve robotic reasoning systems based on graph representation. However, this direction is less explored by the previous research, which focuses on human-robot interaction (HRI) and *how* to interact, control, or supervise a robotic system in real-time, but less on data-centric sense-making of *why* the robot makes a decision. However, prior explorations in the domain of eXplainable AI (XAI) and Visualization [17, 19] suggest that it is promising to develop interfaces that could support users' sense-making of robots' decisions, especially in real-world environments.

To address this under-explored opportunity, we designed and implemented an interface which allows expert-users to ask questions and send commands to the robot in natural language. The interface shows the decision-making process of the robot, based on the knowledge representation in real-time. Furthermore, users can also modify the graph on the fly if a mistake is found. After several weeks of using the system in the experts' daily development of the robotic reasoning system, insights were summarized based on the feedback of the users.

In the remainder of this paper, we will first give an overview of related work in Section 2. Then we briefly introduce the back-end (Section 3), followed by the front-end (Section 4.2) and results of a small user study 5. Finally, we conclude the paper and give an outlook in Section 6.

2 RELATED WORKS

In the domain of Human-Robot interaction, the area of interactive and real-time visualizations for depicting robot decisions on real-world tasks is unfortunately not explored yet. However, there is a large number of interfaces proposed in the HRI domain regarding

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '23, April 23–28, 2023, Hamburg, Germany
© 2023 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9422-2/23/04.
<https://doi.org/10.1145/3544549.3585886>

situational awareness (SA) and control [17, 20]. For example, many teleoperating-robot interfaces provide both the map information and video stream from the camera of the robots [17]. Some of them even fused the 2D camera image into the 3D map [12, 25]. In-depth levels of visualization in such interfaces are to include the sensing, perception, prediction, planning and execution information for showing the internal state and goals of the intelligent agents [6, 16, 23]. However, the aim of such interfaces are usually to provide a supervision feature, instead of visualizing the internal reasoning or intervene if necessary in the decision process.

In contrast, in the data visualization community, many studies and interfaces have been proposed for sense-making. Especially, the knowledge graph is seen as a tool for enhancing the transparency and explainability of an intelligent system [4, 9, 18], which has been applied for many domains, such as visual debugging [15] or recommendation system [11]. There are many visualization designs proposed for the knowledge graph. For example, Force Simulation [7], Force-Directed Edge Bundling [8] and Arc diagram [22] etc. However, up to our knowledge, such kind of interfaces have not been investigated yet in the robotic domain for two main reasons: Firstly, visualizations are used for data inspection, instead of interacting with the system by modifying processes or content on the fly. Especially, robotic developers need such a feature, as they require to change the behaviour of the agent immediately. Secondly, it is important to link the abstract knowledge to the concrete instances in the authentic environment where the robot is operating. Therefore, there is a high demand to adapt the knowledge graph for usage by robotic experts.

3 KNOWLEDGE REPRESENTATION AND SYSTEM STRUCTURE

In this section, we briefly describe the overall system including its components. The core of the system is a Knowledge Engine (KE) that acts as central component (see Figure 1). It consists of a knowledge graph according to MemNet [3] plus reasoning methods operating on the graph. Attached to the KE is a simulator that allows an interaction with a 3D environment, using the knowledge and reasoning capabilities of the KE. The simulator we use is VirtualHome [13], where we added a new room environment that is true to scale with our lab. VirtualHome provides an easy interface to control one or multiple agents by the actions walk/run to, walk forward, turn left/right, grab/put, open/close, switch on/off, touch and look at. We use a single robotic agent only, with combined atomic actions for bring (walk to + grab + put) and go (walk to). Further actions are straightforward, nevertheless, have not been implemented at the state of this experiment. The Semantic Parsing allows to translate natural language input (written text in a chat-box) to the previously mentioned actions or giving answers to questions about the environment of type locating ("where is [object]"), counting ("how many [object] are on [location]") or enumerating ("what is on [location]"). The challenge for the KE is to resolve the instances in the environment from indirect references or synonyms in the language input. Additionally, we extracted commonsense knowledge from ConceptNet about objects, locations or tools that can be used for a certain action, as described in [10]. This commonsense information is inserted into our KE (see Knowledge Insertion in

Figure 1) as so called *action patterns*. The XAI component allows for an intuitive visualization of the internal processes and is the focus in this paper. For the interaction between KE and XAI, it is important to have an understanding of the high level KE access, which is the Semantics Abstraction Layer (SAL), as described in [1, 2].

The SAL is based on getting or setting objects, locations, actions, states (object properties or spatial relations), tools or combinations of those as action patterns. All arguments can be either forwarded as a textual utterance (lemma) or a unique concept ID in the knowledge graph. Further distinction is made between abstract concepts and short term (STM) instances that are attached with properties. We focus on the most important API calls for this paper, which are the STM retrieving functions *get_stm_objects*, *get_stm_locations*, *get_stm_actions*, *get_count*. The function share the arguments *action*, *object*, *location* and *state*. These arguments can be of type lemmas or concept IDs. For each concept, the internal reasoning will also explore its child concepts until they finally hit a matching instance of the environment. That means, we could alternatively identify a *banana* by calling *get_stm_objects* (*object*="fruit", *state*="yellow") or the location of a *banana* by *get_stm_locations* (*object*="fruit", *state*="yellow").

As return value, the functions return always unique concept IDs, which again can be forwarded to any other function, so that we can create a tree of nested calls. Such trees provide an abstract access to the KE reasoning and is the basis for the user interface.

4 THE USER INTERFACE

Before stepping into details of the front-end, we summarize the requirements we like to meet. These requirements are based on feedback of expert users concerning the initial workflow for investigating and developing our system.

4.1 Expert user requirements

In the beginning of the project, the system inspection was quite distributed and uncomfortable. The expert user had to open different applications to start and interact with the system: including opening the VirtualHome application to initiate the robotic simulator; open multiple Linux consoles¹ to start the Knowledge Engine and yet another visualization to crawl the huge knowledge graph. Therefore, we designed and implemented a front-end that combines the distributed system access points into a single web-page, as well as a transparent and efficient visualization of internal decision processes. In a first step, we collected requirements from experts' point of view for a holistic interface:

- (1) sending natural language command via chat-box
- (2) trace internal reasoning process and data structure
- (3) communicate on basic concepts of the KE, not on raw data level
- (4) interface should be accessible from different hosts within a network
- (5) modify existing knowledge in the knowledge graph
- (6) insert new knowledge via chat-box

¹https://en.wikipedia.org/wiki/Linux_console

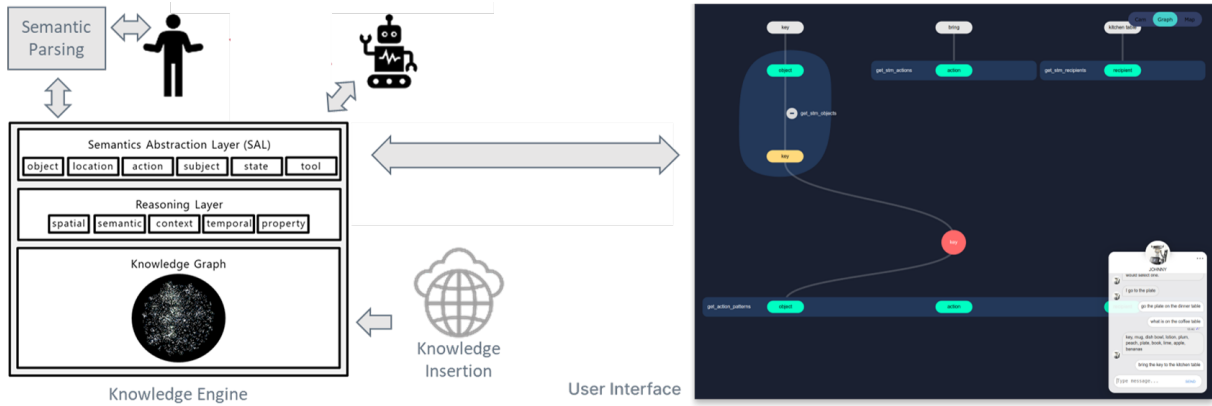


Figure 1: Overall system sketch with the Knowledge Engine as core component. The simulated environment and the semantic parsing allows for accessing the Knowledge Engine by natural language. The User Interface facilitates tracing of the reasoning steps and content of the Knowledge Engine.



Figure 2: User accessing the interface from tablet (left) and PC (middle) or for inspection only on a video panel (right). The interface provides three modes: camera mode (left), map mode (middle) and camera mode (right). On each device, the user can switch between the different modes, as required.

In this paper we concentrate on items 1-4, touch a bit item 5 and leave a more detailed study of 5 and 6 for future work.

4.2 Interface Components

The graphical user interface (GUI) has been designed and implemented as a web-based front-end. Driven by the back-end, expert users can visit the interface anywhere in the research lab with any device (Figure 2). With this we meet already the requirement 4 of our list in Section 4.1. The chat-box interface (see Figure 3.a) allows the user to type in natural language commands and receive answers from the KE, which is in line with requirement 1. By clicking on the toggle button (see Figure 3.b), the user can switch between camera-mode (Figure 3 left), graph-mode (Figure 4) and map-mode (Figure 5), which provides a holistic approach to inspect the system through different access points. In the following sections, we introduce each more in detail.

4.2.1 Camera mode. In the camera mode, the video streaming of the simulator first-person view is displayed (see Figure 3, which allows the user to supervise the robot's behavior in real-time. This mode highlights objects in the scene that are part of a conversation in the chat-box. As the example in Figure 3 shows, after the robot received the command "go to the table", the robot asks the user to

specify whether it is the table in the kitchen or the living room, and then it will go to the table. And the table is highlighted.

4.2.2 Graph Mode. The graph mode visualizes the called sequence of functions and the corresponding input/output. This is already a first step into requirement 2 and demonstrates the robot queries to the KE, based on the language input and the final output after all functions have been processed. As shown in Figure 4, after the user asks "where is something to drink?", the Semantic Parsing component converts the sentence into the "get_stm_objects" function (Figure 4.b) in the first level. Furthermore, the module also extracts the required inputs: "something" (white node, Figure 4.a) as "object" argument (green node, Figure 4.c) and "drink" as "action" argument. The output of the first level function is visualized below the function bar (shown as red nodes, Figure 4.d). As we have a nested function call in this example, the function results are at the same time the input of the next called function "get_location". This function reasons about the location of the object in the environment. Finally, the final results are the "kitchen", "fridge" and "dinner table", which are the locations of objects that match with "something to drink".

If the user clicks the bar of the first level function, it will expand into a hull and show details about the knowledge graph structure (Figure 4.e), as simplified excerpt of the actual raw data. The yellow

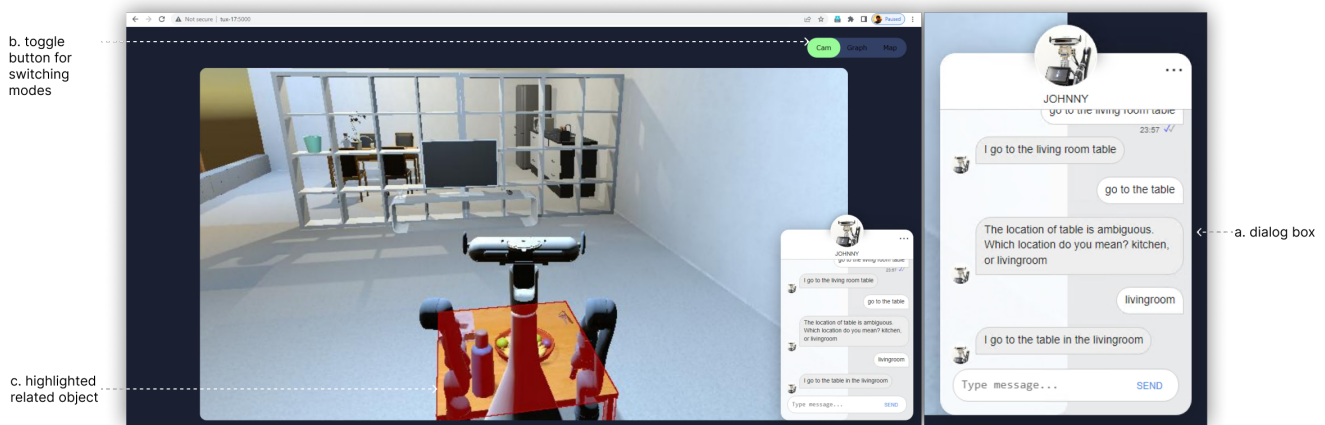


Figure 3: The camera mode, showing the simulation in first-person view as video stream (left). Objects are highlighted in the simulator if they are part of the conversation in the chat-box (either by user or answer of the agent) or if the agent considers them for executing an action.

nodes is abstract knowledge in the KE, which also groups synonyms that express the same concept. The lines from the utterances (white nodes) and parameters (green nodes) to the yellow node (Figure 4.h) indicate that the system found the concepts which have the corresponding utterance attached. As in the user's question, the word "to drink" describes the noun "something", and the "get_stm_objects" function tries to find the parent concepts for "drink" and "something" and if they are related along a certain path in the knowledge graph. This mechanism of searching is visualized: The highlighted red links demonstrate that the instance object juice (red circle) is an instance of the abstract concept juice, which is the child of "food-stuff", "food", "substance", "matter" and finally "something"; and it is also connected from "juice" further to "drink".

As the graph data in our knowledge engine is extracted from commonsense knowledge, therefore it may contain errors, which cause problems in a robotic setting, if it cannot be corrected. According to the requirement 5 in Section 4.1, in the graph mode, people can select and highlight the links for inspection. Furthermore, by pressing the "delete" button on the keyboard, the highlighted links will be deleted from the database. This changes the results of the answers by excluding the selected knowledge from reasoning.

The graph is implemented in a force simulation style of d3.js². All the yellow nodes are dragged from a central gravity point which is the folding-button (4.f) position, which makes them clustered; At the same time, each of the nodes also has a collider to push them away from overlapping. Users can freely drag each node to put it in a preferred position. But after the user drop the node, its position will be updated by the gravity and colliders.

4.2.3 Map mode. The map mode shows a 3D map of the simulator environment to provide a holistic view for linking the abstract knowledge graph and the scene in which the robot is operated. The 3D map contains the models of all the static objects in the scenario

in a low-poly³ style, which ignores the details of the real objects, for providing a more concise view to the users. Above the map, a 3D graph visualization is provided. The 3D graph has exactly the same nodes and links with the 2D graph in the graph mode. But the instance node is located according to the real object's position in the 3D map (5.b). In the map mode, the user can zoom in/out and rotate the view by mouse to observe the map and the 3D graph.

5 USER FEEDBACK AND REFLECTIONS

This interface was implemented in the lab environment at our research lab and used 5 robotic experts (males, 35-45 years old) in their daily development on the system. After 3 weeks of usage, a number of insights were collected from the interview among the expert users. The developer observations and the suggestions have been collected as Gitlab issues⁴ on a voluntary basis. In the following sections, we summarized the insight and reflections.

5.1 Finding and deleting the wrong links

One of the most important features of the interface is finding the wrong links in the database. As reported by one user, after he tried to ask "where is something to eat?" the robot answered that "fork" and "salmon" can be found on the "dining room table" and in the "fridge" correspondingly. However, forks are not food. After expanding the hull in the graph mode, the user traced from the fork instance back to the eating action. Then he found there is a link between "eat" and "fork", which indicated that fork was seen as eatable in the knowledge graph. This mistake was directly corrected by deleting this link in action, and the fork was not referred to afterwards.

Insights and suggestions: The deleting feature was highly appreciated by the developers, however, they also asked to improve the graph visualization and the interactions. Firstly, there are many nodes spread randomly after the hull expanded, even though they

²<https://github.com/d3/d3-force/tree/v3.0.0>

³https://en.wikipedia.org/wiki/Low_poly

⁴<https://about.gitlab.com/>

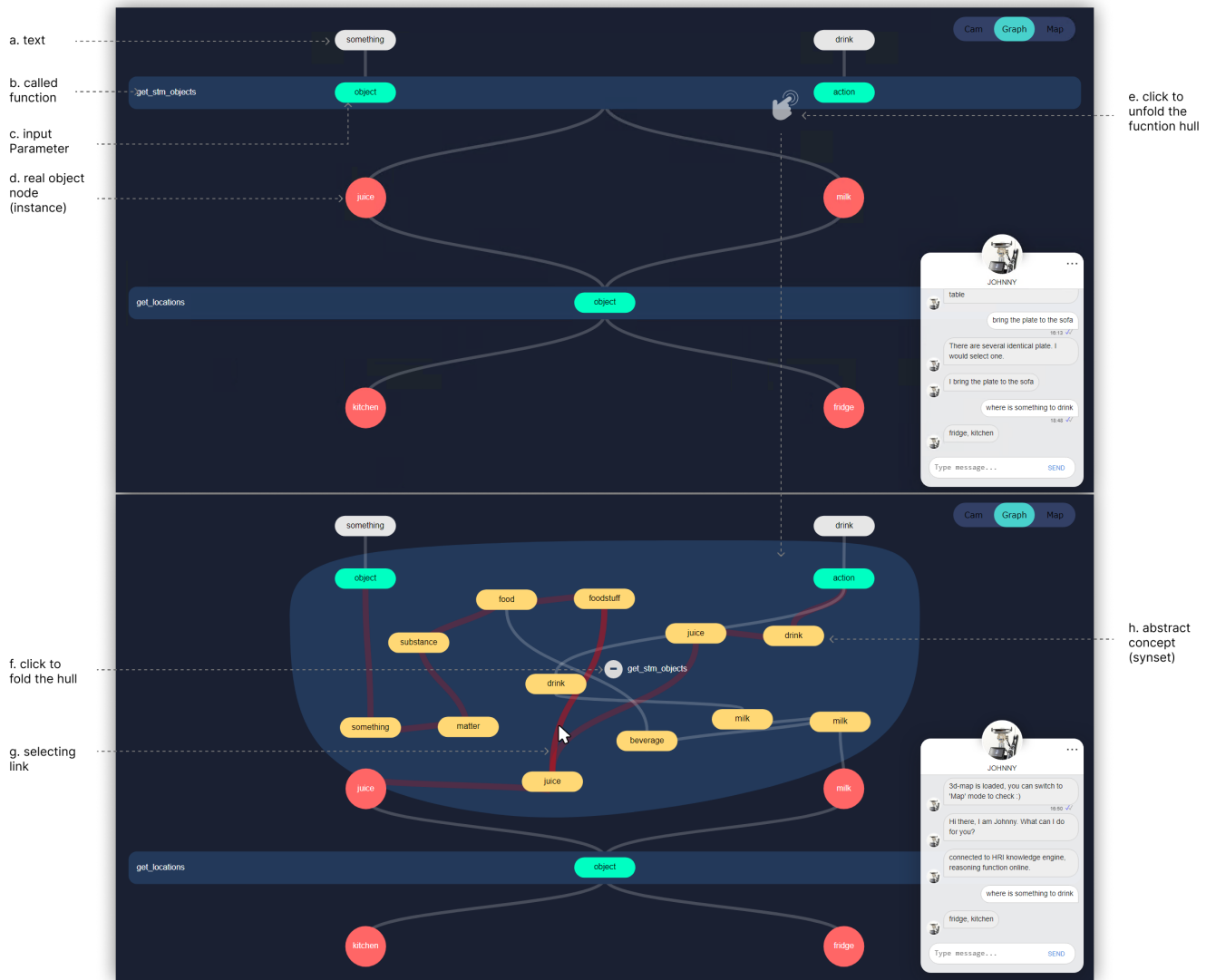


Figure 4: The graph mode showing the order of executed functions (top) and, in an extended view, the connected data and paths included in the reasoning.

are attracted by the centre gravity and collide with each other without overlapping, the graph still looks chaotic if too many nodes are returned. In that case, the user needs to manually arrange the position of the nodes. It is suggested to put the nodes in different layers according to the parent-child hierarchy. Secondly, users suggested the feature of highlighting related links, when the instance node is clicked. Thirdly, they also suggested to have a confirmation button or recovery button after a certain link is selected for deleting. Sometimes, either the user accidentally deletes wrong links, or they want to check how the deleted link influences the answer of the robot.

5.2 Referred objects and attributes

Applying a knowledge representation on a robotic system requires the abstract concept to be linked to the concrete object instance in the environment. Our interface presents this kind of grounding by highlighting a related object in the camera mode and showing the position of the object in the 3D map. A user reported these features helped them find the mismatch between the simulator environment and the knowledge graph regarding some objects' attributes. Another example is that one user asked for "bring the key to the kitchen". then the robot executed this command. Strangely, when the user asks "where is the key?", the robot answers 2 positions. After checking the map and asking for the number of the key in

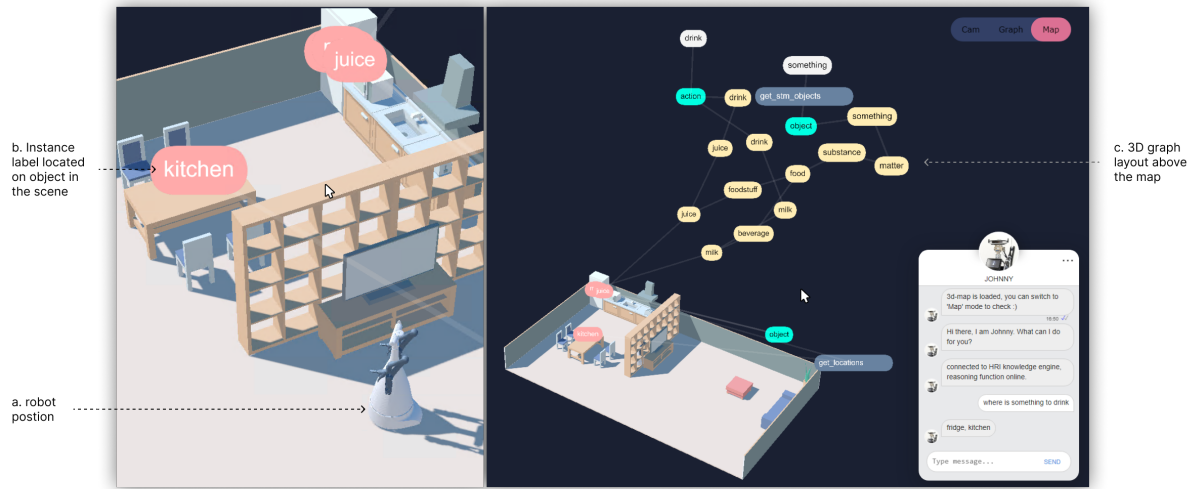


Figure 5: The map mode. Right: the 3D map and 3D graph visualization. Left: user can zoom in and rotate the view to observe the map.

the kitchen and living room, the user found the bug of the knowledge engine that it duplicated certain instance object after the state (position) changed.

Insights and suggestions: Users found that highlighting objects and positioning instance nodes in the 3D map is useful for locating referred objects. Besides this, users suggested fusing up to some extent the camera view and graph view, to reduce too many switching between view. Another feedback is that the chat-box which allowed users to ask questions forth and back is extremely important for identifying issues. It made the developer feel like debugging the system via natural language, which increases the efficiency and enhances the experience. At least the identification on high-level helped to find an entry point much faster, even though they might have to go back to the console or database for in-depth checking the problems sometimes.

5.3 Visualization of called function sequence

The feature of displaying called function sequences was welcomed by the users, as it provides a clear view of the function name, parameter and input/output, which can be easily understood by the experts. One user reported he found the bugs of the wrong-called function by reading the function name on the hull. Three users reported they found the problem that certain functions could not return correct output with the help of the visualization.

Insights and suggestions: The visualization of the called functions explicitly demonstrates the system's decision-making process, which got high acceptance among the developers. But some improvements were suggested: sometimes the problems come from the semantic parsing component, which may fail to translate the natural languages into correct functions to call. Some users also want to see this procedure; and currently, the utterance of the output is shown, it would be better to also show concept IDs for further inspection in the knowledge graph.

6 CONCLUSIONS AND FUTURE WORKS

Nowadays, an increasing number of researchers utilize graph representation to enhance the capability of robots at the reasoning level. However, the advantage of graph representations in the perspective of transparency and explainability has not been well reflected in the current HRI interfaces and more design exploration for real-time and contextual, tailored tracing is an urgent need. To address this opportunity, we designed a processing-centric interface that allows the robotic expert to interact with the robot, make sense of the decision process and even modify the knowledge presented on the fly. After being implemented in the lab and used by expert users in their daily work, they found the interface useful in supporting them to resolve problems in the system and database. The next step of this research is to improve the interface according to the feedback from users and to conduct a long-term research for more qualitative results. As new features, we like to tackle items 5 and 6, as listed in Section 4.1, by the ability to further modify the graph by also deleting, inserting or merging action patterns, as well as inserting new action patterns or concepts via speech.

REFERENCES

- [1] Joerg Deigmoeller., Pavel Smirnov., Viktor Losing., Chao Wang., Johane Takeuchi., and Julian Eggert. 2022. Situational Question Answering using Memory Nets. In *Proceedings of the 14th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management - KEOD*. INSTICC, SciTePress, 169–176. <https://doi.org/10.5220/0011549200003335>
- [2] J. Eggert, J. Deigmoeller, L. Fischer, and A. Richter. 2020. Action Representation for Intelligent Agents Using Memory Nets. In *Communications in Computer and Information Science*. SPRINGER.
- [3] Julian Eggert, Jörg Deigmöller, Lydia Fischer, and Andreas Richter. 2019. Action Representation for Intelligent Agents Using Memory Nets. *International Joint Conference on Knowledge Discovery, Knowledge Engineering, and Knowledge Management*, 260–282.
- [4] Leuce F. 2020. On the Role of Knowledge Graphs in Explainable AI. *Semantic Web* 1 (2020), 1–5. Issue 0. <http://semantic-web-journal.org/system/files/swj2259.pdf>
- [5] Jan Philip Göpfert, Ulrike Kuhl, Lukas Hindemith, Heiko Wersing, and Barbara Hammer. 2021. Intuitiveness in Active Teaching. *IEEE Transactions on Human-Machine Systems* 52, 3 (2021), 458–467.

- [6] Shashank Govindaraj, Keshav Chintamani, Jeremi Gancet, Pierre Letier, Boris van Lierde, Yashodhan Nevatia, Geert De Cubber, Daniel Serrano, Miguel Esbri Palomares, and Janusz Bedkowski. 2013. The icarus project-command, control and intelligence (c2i). *2013 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 1–4.
- [7] Jeffrey Heer, Stuart K Card, and James A Landay. 2005. prefuse: A Toolkit for Interactive Information Visualization.
- [8] Danny Holten and Jarke J. Van Wijk. 2009. Force-Directed edge bundling for graph visualization. *Computer Graphics Forum* 28 (2009), 983–990. Issue 3. <https://doi.org/10.1111/j.1467-8659.2009.01450.x>
- [9] Takahiro Kawamura, Shusaku Egami, Koutarou Tamura, Yasunori Hokazono, Takanori Ugai, Yusuke Koyanagi, Fumihito Nishino, Seiji Okajima, Katsuhiko Murakami, Kunihiro Takamatsu, Aoi Sugiura, Shun Shiramatsu, Shawn Zhang, and Kouji Kozaki. 2019. Report on the first knowledge graph reasoning challenge 2018 - toward the eXplainable AI system -. *arXiv* (2019), 1–18.
- [10] V. Losing, L. Fischer, and J. Deigmoeller. 2021. Extraction of Common-Sense Relations from Procedural Task Instructions using BERT. In *Proceedings of the 11th Global Wordnet Conference*.
- [11] Weizhi Ma, Woojeong Jin, Min Zhang, Chenyang Wang, Yue Cao, Yiqun Liu, Shaoping Ma, and Xiang Ren. 2019. Jointly learning explainable rules for recommendation with knowledge graph. *The Web Conference 2019 - Proceedings of the World Wide Web Conference, WWW 2019* (2019), 1210–1221. <https://doi.org/10.1145/3308558.3313607>
- [12] Curtis W. Nielsen and Michael A. Goodrich. 2006. Comparing the usefulness of video and map information in navigation task. *HRI 2006: Proceedings of the 2006 ACM Conference on Human-Robot Interaction 2006* (2006), 95–101. <https://doi.org/10.1145/1121241.1121259>
- [13] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. 2018. Virtualhome: Simulating household activities via programs. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 8494–8502.
- [14] Bernhard Sendhoff and Heiko Wersing. 2020. Cooperative intelligence-a humane perspective. In *2020 IEEE International Conference on Human-Machine Systems (ICHMS)*. IEEE, 1–6.
- [15] Hendrik Strobelt, Sebastian Gehrmann, Michael Behrisch, Adam Perer, Hanspeter Pfister, and Alexander M. Rush. 2019. Seq2seq-Vis: A Visual Debugging Tool for Sequence-to-Sequence Models. *IEEE Transactions on Visualization and Computer Graphics* 25 (2019), 353–363. Issue 1. <https://doi.org/10.1109/TVCG.2018.2865044>
- [16] Daniel Szafr, Bilge Mutlu, and Terrence Fong. 2017. Designing planning and control interfaces to support user collaboration with flying robots. *International Journal of Robotics Research* 36 (6 2017), 514–542. Issue 5-7. <https://doi.org/10.1177/0278364916688256>
- [17] Daniel Szafr and Danielle Albers Szafr. 2021. Connecting human-robot interaction and data visualization. *ACM/IEEE International Conference on Human-Robot Interaction*, 281–292. <https://doi.org/10.1145/3434073.3444683>
- [18] Ilaria Tiddi and Stefan Schlobach. 2022. Knowledge graphs as tools for explainable machine learning: A survey. *Artificial Intelligence* 302 (1 2022), 103627. <https://doi.org/10.1016/j.artint.2021.103627>
- [19] Chao Wang and Pengcheng An. 2021. Explainability via Interactivity? Supporting Nonexperts’ Sensemaking of pre-trained CNN by Interacting with Their Daily Surroundings. In *Extended Abstracts of the 2021 Annual Symposium on Computer-Human Interaction in Play*. 274–279.
- [20] Chao Wang, Stephan Hasler, Manuel Mühlig, Frank Joublin, Antonello Ceravola, Joerg Deigmoeller, Lydia Fischer, and Pengcheng An. 2021. Designing interaction for multi-agent cooperative system in an office environment. In *Companion of the 2021 ACM/IEEE International Conference on Human-Robot Interaction*. 668–669.
- [21] Xiang Wang, Dingxian Wang, Canran Xu, Xiangnan He, Yixin Cao, and Tat Seng Chua. 2018. Explainable reasoning over knowledge graphs for recommendation. *arXiv* 33, 5329–5336. Issue 01.
- [22] M. Wattenberg. 2002. Arc diagrams: Visualizing structure in strings. *Proceedings - IEEE Symposium on Information Visualization, INFO VIS 2002-January*, 110–116. <https://doi.org/10.1109/INFVIS.2002.1173155>
- [23] Matt Whitlock, Keke Wu, and Danielle Albers Szafr. 2020. Designing for Mobile and Immersive Visual Analytics in the Field. *IEEE Transactions on Visualization and Computer Graphics* 26 (2020), 503–513. Issue 1. <https://doi.org/10.1109/TVCG.2019.2934282>
- [24] Yikun Xian, Zuohui Fu, S. Muthukrishnan, Gerard De Melo, and Yongfeng Zhang. 2019. Reinforcement knowledge graph reasoning for explainable recommendation. *SIGIR 2019 - Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (2019), 285–294. <https://doi.org/10.1145/3331184.3331203>
- [25] Holly A. Yanco, Brenden Keyes, Jill L. Drury, Curtis W. Nielsen, Douglas A. Few, and David J. Bruemmer. 2007. Evolving interface design for robot search tasks. *Journal of Field Robotics* 24, 779–799. Issue 8-9. <https://doi.org/10.1002/rob.20215>